

- 《高级AI应用开发课：以模型能力为核心的软件工程新范式》课程课纲（26课）
 - 课程定位
 - 适合人群
 - 学习主线
 - 一、总排期概览
 - 二、课程实操项目
 - 项目名称
 - 项目目标
 - 课程统一案例
 - 推荐最终结构
 - 学习与实践安排
 - 三、参考案例与学习环境
 - 参考案例与参考工具
 - 参考案例与工具在课程中的应用
 - 核心模块源码 / 架构解析范围
 - 单章学习结构
 - 环境准备范围
 - 课程组织方式
 - 四、详细课纲
 - 第 0 章：入门、参考案例、环境准备与系统全貌（2 课）
 - 第 1 章：单次请求、任务执行器与 TriggerFlow 入门（2 课）
 - 第 2 章：规划、决策、行动与结构化输出（2 课）
 - 第 3 章：工具 / MCP 接入（2 课）
 - 第 4 章：执行面与沙盒（2 课）
 - 第 5 章：消息入口、会话归属与 Gateway 控制面（2 课）
 - 第 6 章：Runtime 组装与运行时引擎（3 课）
 - 第 7 章：Skills 类型、供应与消费机制（3 课）
 - 第 8 章：事件流、流式反馈与主动任务（2 课）
 - 第 9 章：状态管理、可控记忆与任务恢复（2 课）
 - 第 10 章：Eval、回归测试与可观测性（2 课）
 - 第 11 章：成本、可靠性、安全基线与服务化收束（2 课）
 - 五、课程组织路径
 - 1. 案例引入、原理讲解、模块实践与系统集成
 - 2. 参考案例与课程项目双线并进
 - 3. 环境准备与框架初始化前置
 - 4. 从单次请求到基础编排，再逐步扩展
 - 5. 按具体技术栈推进系统搭建

- 6. 企业内部场景贯穿全程
- 六、课程交付成果

《高级AI应用开发课：以模型能力为核心的软件工程新范式》课程课纲（26课）

课程定位

- 课程以 OpenClaw 作为主要参考案例，并结合 Codex、Claude Code 等模型原生系统展开模块对照。
- 课程内容覆盖单次请求、编排、工具层、沙盒、Runtime、Skills、事件流、状态管理、可控记忆、评测回归、观测、成本与可靠性控制等关键能力。
- 课程目标是帮助学员建立对模型能力驱动软件系统的整体认知，并基于 **Agently ModelRequest + Agently TriggerFlow + FastAPI + FastMCP** 搭建一套可服务化、可扩展的通用智能助理服务原型。

适合人群

- 具备一定编程基础，希望在 AI 时代提升检索、整理、生成、分析和流程协同能力的技术型从业者。
- 希望理解 AI 能力如何从个人提效工具发展为可复用、可服务化系统能力的人。
- 希望理解 AI 对软件设计、开发流程、系统架构和工程协作方式带来哪些变化的人。
- 希望从应用开发进一步成长为高级开发工程师、AI 应用工程师、架构师或技术负责人的人。
- 更适合具备基础 **Python**、脚本开发、API 调用或 Web 服务经验的学员。

学习主线

多线并进：

- 通过参考案例与原理分析，掌握模型原生系统从入口、决策、工具、沙盒到状态管理的整体结构。

- 通过 OpenClaw 的完整演示，以及 Codex、Claude Code 的模块对照，掌握不同智能体系统的共性能力与关键差异。
- 通过源码与架构解析，掌握 界面 / 行为现象 -> 系统模块边界 -> 关键代码链路或架构切片 的分析方法。
- 通过 OpenClaw 安装运行、Codex / Claude Code 演示，以及 Agently ModelRequest、Agently TriggerFlow、FastAPI、FastMCP 的实际搭建，掌握课程使用的核心工具与工程栈。
- 通过通用智能助理服务的模块实践与系统集成，掌握 Agently ModelRequest 请求层、TriggerFlow 编排层、工具层、MCP Server、HTTP API、事件流与状态存储的具体实现方式。
- 通过服务化架构设计，掌握配置分层、Gateway 与执行模块拆分、MCP 与工具封装、状态分层、可控记忆、观测与治理等工程方法。

一、总排期概览

章	课数	主题	课程项目新增的能力
0	2	入门、参考案例、环境准备与系统全貌	完成 OpenClaw 安装运行、Codex / Claude Code 演示，搭起 Agently + TriggerFlow + FastAPI 最小工程骨架
1	2	单次请求、任务执行器与 TriggerFlow 入门	用 Agently ModelRequest 完成单次请求，并用 Agently TriggerFlow 搭建最小编排流程
2	2	规划、决策、行动与结构化输出	增加结构化决策与 action planner
3	2	工具 / MCP 接入	接入本地工具、HTTP 工具，并用 FastMCP 封装 MCP Server
4	2	执行面与沙盒	增加受限命令 / 脚本执行能力
5	2	消息入口、会话归属与 Gateway 控制面	用 FastAPI 暴露任务 API、SSE 与 session / route
6	3	Runtime 组装与运行时引擎	在 Agently ModelRequest 外组装自定义 runtime 模块

章	课数	主题	课程项目新增的能力
7	3	Skills 类型、供应与消费机制	建立多类型 Skill 资产，以及 Skills 解析与接入模块
8	2	事件流、流式反馈与主动任务	用 Agently TriggerFlow 增加过程事件流和主动任务
9	2	状态管理、可控记忆与任务恢复	增加 session、显式资料写回、checkpoint / resume
10	2	Eval、回归测试与可观测性	增加样例集、回归检查、trace / replay
11	2	成本、可靠性、安全基线与服务化收束	增加预算、超时重试、最小安全与部署收束

合计：26 课

二、课程实操项目

项目名称

- 通用智能助理服务

项目目标

- 接收通用任务，如问答检索、信息整理、文档生成、表单填写、规则查询、简易多步任务执行。
- 能通过工具或 MCP 访问企业内部接口，获取业务数据、审批状态、知识内容和流程信息。
- 能把 Skills、工具和 TriggerFlow 编排组合起来，形成企业内部可复用的智能工作流。
- 能够规划动作、调用工具、按需进入受限执行环境、流式回传过程、保存会话状态，并对经确认的资料、偏好和任务状态进行显式写回。
- 能记录关键事件、执行轨迹、成本与失败原因，并对高风险动作走审批。
- 统一案例聚焦企业内部通用智能助理服务的基础能力与系统实现。

- 课程实操默认采用 **Agently ModelRequest + Agently TriggerFlow + FastAPI + FastMCP** 作为实现基础。

课程统一案例

- 企业内部智能问答与信息检索：查询制度、流程、知识库内容，补充缺失信息并生成可执行答复。
- 企业内部任务管理与流程协同：收集字段、生成申请单 / 登记单、查询审批状态、生成待办与提醒。
- 长文与资料梳理：整理会议纪要、制度文件、项目周报、汇报材料，输出摘要、结论和行动项。
- 内网接口访问与数据取用：通过工具或 MCP 访问企业内部接口，查询人员信息、审批进度、业务指标或基础资料。
- 内部智能工作流：通过 TriggerFlow 编排多步流程，并用 MCP + Skills 封装复用能力。

推荐最终结构

- **gateway-service**：基于 **FastAPI**，负责 API 入口、鉴权、session / route、SSE / WebSocket 推送。
- **agent-core**：基于 **Agently ModelRequest** 和自定义模块，负责任务执行器、planner、runtime、状态推进。
- **tool-adapters**：负责本地工具、HTTP 工具、MCP 工具和内网接口能力的统一封装。
- **mcp-servers**：基于 **FastMCP**，负责把审批查询、知识检索、业务数据访问等能力封装为 MCP Server。
- **skills-registry**：负责 Skills 封装、版本管理、能力组合与业务复用。
- **flow-orchestrator**：基于 **Agently TriggerFlow**，负责多步任务路由、主动流程与流程级状态推进。
- **sandbox-worker**：负责命令、受限脚本、浏览器等高风险执行能力。
- **memory-store**：负责短期状态、用户资料、显式写回记录与任务恢复点。
- **eval-observer**：负责样例集、回归检查、trace、回放与质量观测。
- **ops-center**：负责配置、预算、审批、日志、审计与运行参数管理。

学习与实践安排

- 每节课约 2 小时。
 - 每一章按“参考案例引入 -> 概念与原理 -> 源码 / 架构解析 -> 模块实践或深读 -> 集成与服务化边界”推进。
 - 实践环节以“最小可运行、结构清晰、可继续扩展”为主要完成标准。
 - 章节实践默认使用 **Python + Agently ModelRequest + Agently TriggerFlow + FastAPI + FastMCP**。
-

三、参考案例与学习环境

参考案例与参考工具

- **OpenClaw**：用于安装、运行、配置、日志查看、任务执行过程演示和关键架构入口解析。
- **Codex**：用于任务分解、代理式执行 workflow、工具调用、验证回路、审批与执行边界等模块对照。
- **Claude Code**：用于模型原生 workflow、上下文组织、Skills / 记忆组织方式、执行面边界、安全限制与协作体验等模块对照。

参考案例与工具在课程中的应用

- 第 0 章完成 OpenClaw 的本地安装、模型接入、基础排障，并演示一次完整任务执行过程。
- 第 0 章完成 Codex / Claude Code 的基础演示环境准备，并分别演示任务分解、工具调用、上下文组织和执行边界。
- 各章内容明确对应参考案例、演示场景和模块主题。
- 第 1 章同时使用 **Agently ModelRequest** 和 **Agently TriggerFlow** 搭建单次请求与最小编排流程。
- 第 2 至第 6 章在 **Agently ModelRequest + Agently TriggerFlow** 基础上逐步扩展 planner、tool runtime、gateway 与 runtime 模块。
- 第 7 章对照 Codex / Claude Code 的 Skills 机制，分别实现 reference 型、script 型、MCP 依赖型 Skill。
- 第 3 章使用 **FastMCP** 封装一个企业内部 MCP Server。
- 第 5 章使用 **FastAPI** 暴露任务提交、结果查询与 SSE 事件流接口。
- 第 8 章使用 **Agently TriggerFlow** 实现定时提醒、审批跟进和主动任务流程。

- 第 9 章重点落在 session、用户资料、经确认资料写回和任务恢复，不把“自动长期记忆”作为默认落地方案。
- 关键主题结合“案例行为 -> 模块边界 -> 源码 / 架构切片”展开分析。

核心模块源码 / 架构解析范围

- 入口与控制面：消息入口、route、session、鉴权、配置装配。
- 单次任务执行过程：turn、attempt、planner、executor、fallback、状态推进。
- 工具层：tool registry、tool adapter、MCP client / server、内网接口封装、错误回传路径。
- 能力封装层：reference 型 Skill、script 型 Skill、MCP 依赖型 Skill、能力组合、依赖关系、版本管理。
- 编排层：TriggerFlow、流程节点、流程状态、事件驱动衔接。
- 执行面：sandbox worker、命令执行、受限脚本、文件边界、权限与审批。
- Runtime：context builder、prompt assembly、subscriber / hooks、event dispatch。
- 状态层：transcript、session store、profile store、checkpoint、显式写回、replay。

单章学习结构

- 案例演示：呈现结果、过程、日志或界面变化。
- 原理讲解：梳理概念、状态和模块边界。
- 源码 / 架构解析：定位核心链路和关键接口。
- 模块实践：完成实现、深读或集成。

环境准备范围

- 参考工具环境：OpenClaw 安装运行、Codex / Claude Code 演示环境、账号 / Key / 模型接入、配置文件、日志入口与最小排障。
- 课程实操环境：Python + Agently ModelRequest + Agently TriggerFlow + FastAPI、状态存储、最小任务台、日志目录、eval 样例目录。
- 章节专项环境：FastMCP Server、Sandbox Worker、SSE / WebSocket 事件流出口、profile / checkpoint 存储、回归检查任务。

课程组织方式

- 第 0 章完成 OpenClaw、Codex、Claude Code 演示环境，以及 **Agently ModelRequest + Agently TriggerFlow + FastAPI** 实操环境的启动与验证。
- 后续各章从案例演示切入，逐步进入概念、原理和实现。
- 参考案例中的外显行为对应到系统模块、状态推进和服务边界。
- 关键模块配套源码 / 架构解析。

四、详细课纲

第 0 章：入门、参考案例、环境准备与系统全貌（2 课）

参考场景：企业制度查询答复、内部门户表单助手、会议纪要与周报整理

节（每节课约 2 小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
第 0-1 课：开班典礼 & 模型能力驱动的软件工程新范式	1. 从模型能力持续提升出发，讨论软件工程在需求理解、任务执行、工具调用、编排组织、交付方式上的变化；2. 结合 OpenClaw、Codex、Claude Code 的典型表现，说明为什么这一轮变化不只是“多了一个 AI 工具”，而是在重写软件系统的组织方式；3. 说明课程目标、学习路径与最终交付；4. 建立“控制面 + 执行面 + 状态层”的整体认知，并把学员认知引入后续课程	明确参考案例边界与课程项目边界，画出通用智能助理服务最终蓝图	学员建立课程坐标系，理解模型能力提升带来的工程范式变化，以及课程内容、系统范围与学习主线	从产品现象进入系统结构，建立对模型驱动软件工程的整体认知框架
第 0-2 课：课程学习环境准备	1. 完成 OpenClaw 安装、模型接入、配置与排障，并演示一次完整任务执行过程；2. 演示 Codex 与 Claude Code 的典型任务过程、配置入口和日志入口；3. 创建课程项	完成 OpenClaw 演示环境、Codex / Claude Code	学员完成参考环境与实操环境启动，明确课程	本地开发环境、模型接入层、配

节（每节课约2小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
	目的最小工程骨架：包含 Agently ModelRequest + Agently TriggerFlow + FastAPI 的空白后端、简易任务台占位、配置文件、日志目录、基础 eval 目录；4. 说明后续各章对应的实现模块与所用框架，后续随课程逐章补全功能	对照演示，并搭起课程项目最小工程骨架	项目将如何边学边搭、逐章补全	置层与业务核心层分离；环境准备与运行验证纳入课程范围

第 1 章：单次请求、任务执行器与 TriggerFlow 入门（2 课）

参考场景：会议纪要整理并生成待办、企业制度问答并补齐缺失信息

节（每节课约2小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
第 1-1 课：Agently ModelRequest 入门与最小任务执行器	1. 以“整理纪要并生成待办”或“根据制度回答问题”为例，展示单次请求完成任务的基本方式； 2. 区分输入、提示词、输出、错误收口； 3. 明确单次请求适合解决什么问题、不适合解决什么问题； 4. 实现一个基于 Agently ModelRequest 的最小任务执行器	使用 Agently ModelRequest 搭建命令行版最小任务执行器，完成 input -> model request -> result	学员掌握单次请求和最小任务执行器的边界与实现方式	先把请求接口、模型参数、日志和错误收口做清楚，不把单次请求逻辑直接绑定到 HTTP 或界面层

节（每节课约2小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
第 1-2 课： Agently TriggerFlow 入门与最小编排流程	1. 从单次请求的边界出发，引入为什么需要编排；2. 讲清 TriggerFlow 的节点、状态、路由和最小流程组织方式；3. 用两步或三步任务展示最小编排流程；4. 说明并发规划、流式输出、主动触发等高级能力会在后续章节展开	将单次请求执行器升级为一个基于 Agently TriggerFlow 的最小编排流程	学员理解为什么课程会把 TriggerFlow 作为后续大部分编排组织的基础方案	编排层与模型请求层分离；流程状态与节点状态显式管理；为后续 planner、tool、event 预留扩展点

第 2 章：规划、决策、行动与结构化输出（2 课）

参考场景：出差申请信息收集、报销问答与下一步建议

节（每节课约2小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
第 2-1 课： 从任务理解到动作决策	1. 以“补齐申请字段 -> 查询规则 -> 生成表单”为例，展示规划、决策与工具调用过程；2. 区分结构化输出与自然语言输出；3. 明确 Function Calling 的作用边界；4. 梳理复杂任务	基于 Agently ModelRequest + Agently TriggerFlow 增加结构化决策层，输出 action schema	学员能把“回答问题”和“驱动动作”区分开，理解结构化决策	输出契约独立于业务逻辑；planner 与 executor 分层，避免“模型输出即控制流”

节 (每 节课 约 2 小时)	课程核心内容	课程项目演进	技能 / 认 知成长 收益	架构思想收 益
	中的任务分解、动作决策与重规划		层的必 要性	
第 2-2 课： 结构 化决 策链 路实 现	1. 设计结构化输出 schema；2. 设计最小 action contract；3. 处理参数校验、执行结果回填与失败兜底；4. 分析显式状态机与隐式模型推理的边界	完成 Agently ModelRequest + planner + executor + TriggerFlow 协作链路	学员掌握模型可执行化的关键能力，不再只停留在文本问答	契约先行、校验先行、失败路径先行；后续工具层才能稳定接入

第 3 章：工具 / MCP 接入（2 课）

参考场景：内网审批状态查询、内部知识库检索、人员与业务信息查询

节 (每 节课约 2 小时)	课程核心内容	课程项目演进	技能 / 认知成 长收益	架构思想收 益
第 3-1 课：工 具发 现、调 用与 MCP 协 同	1. 以审批状态查询、内部知识库检索、人员与业务信息查询为例，展示本地工具、HTTP 工具和 MCP 工具的调用过程；2. 说明工具与 MCP 的角色；3. 区分它们与 Skills 的关系；4. 明确工具接入的治理要求	先接入一个本地工具和一个 HTTP 工具，统一成同一套 tool contract	学员能区分 Tools、MCP、Skills 在系统中的位置，并理解工具接入必须纳入治理	适配层与核心层分离；工具元数据、权限标签、超时策略都要显式定义
第 3-2 课：工	1. 设计工具接口；2. 处理参数校验、返回结构、错	增加一个 FastMCP	学员掌握从“外部系统能	本地工具、HTTP 工

节（每节课约2小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
具封装与 MCP 适配实现	误规范；3. 使用 FastMCP 封装一个企业内部 MCP Server；4. 为后续运行时装配准备统一注册方式	Server，并纳入统一注册表	力”到“智能体可用能力”的封装方法	具、MCP 工具三类接口统一封装；版本、权限、超时、日志统一治理

第 4 章：执行面与沙盒（2 课）

参考场景：内部门户表单填写、批量文件整理、受限脚本处理表格或文本

节（每节课约2小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
第 4-1 课：执行面边界与沙盒职责	1. 以浏览器表单填写、受限命令执行、文件处理为例，展示执行行为与权限边界；2. 明确受限脚本执行、命令执行、文件操作的范围；3. 区分执行面与控制面的职责；4. 明确高风险能力的治理前提	给服务新增一个受限执行器，用于高风险动作	学员理解执行面不是附加功能，而是高风险核心能力	执行面必须和 Gateway、Planner、Runtime 分开部署或至少分开模块
第 4-2 课：最小沙盒设计与完善	1. 文件系统隔离；2. 资源限制；3. 命令与受限脚本执行边界；4. 网络权限控制；5. 审批与回收机制；6. 区分演示环境中的沙盒与真实服务沙盒的差异	为受限执行器补齐工作目录、白名单、超时、清理、简单审批	学员能说清楚一个最小可用沙盒该怎么设计	worker 隔离、权限策略、资源限制和审计记录要先于“把功能跑起来”

第 5 章：消息入口、会话归属与 Gateway 控制面（2 课）

参考场景：企业 IM 入口、内部门户入口、审批或服务台入口

节（每节课约 2 小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
第 5-1 课：入口归属、会话路由与控制面	1. 展示不同入口进入系统后的统一处理方式；2. 说明 agent、session、route 的系统作用；3. 明确归属判定优先于内容生成；4. 区分渠道适配与业务中心	使用 FastAPI 为执行器增加 HTTP 入口，支持 task_id、session_id、route	学员开始理解入口接入不只是“接个聊天渠道”，而是先做归属与控制面	Gateway 负责接入、鉴权、路由、配置，不直接承担执行内核
第 5-2 课：Gateway 与任务台集成	1. Gateway 的职责：接入、鉴权、配置、路由、治理；2. sessionKey / route 的系统价值；3. 简易任务台如何展示任务状态；4. API 入口与前端任务台的最小配合方式	使用 FastAPI 完成任务提交、结果查询、SSE 推送接口，并接入简易任务台	学员知道可服务化系统首先要搭的往往不是 Agent，而是控制平面	API 层、前端展示层、执行内核分离；配置项单独管理，不写死在业务代码里

第 6 章：Runtime 组装与运行时引擎（3 课）

参考场景：查制度 -> 补信息 -> 访问内网接口 -> 输出结果 的多步任务

节（每节课约 2 小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
第 6-1 课： Runtime 的职责与工作方式	1. 展示参考案例中的 turn、attempt、上下文拼装、工具注入与事件订阅过程；2. 明确 prompt 在 runtime 中只是其中一层；3. 区分 runtime 与 workflow 的边界；4. 理解 runtime 在 planner、工具、沙盒与入口之上的装配逻辑	梳理前序模块，确定它们在 runtime 中的装配位置	学员建立“runtime 才是发动机”的认知，理解运行现象与抽象设计的对应关系	Runtime 不是单文件逻辑，而是上下文、工具、事件、状态推进的协同层
第 6-2 课： Runtime 核心组成拆解	1. system prompt builder；2. transcript / context 处理；3. tool runtime；4. subscriber / hooks；5. fallback / retry / compaction；6. 说明这些组成在运行时的协同关系	把 Agently ModelRequest 、planner、tool registry、sandbox、FastAPI Gateway 统一接入 runtime 模块	学员理解一个企业级 Agent Runtime 至少需要哪些模块	配置层、核心 runtime、外部依赖适配层三层拆分；避免把所有逻辑堆进一个 agent 类
第 6-3 课：最小 Runtime 实现与关键链路解析	1. 实现轻量 runtime 原型；2. 让上下文、工具、状态、事件在同一回路中工作；3. 深读或对照样本系统关键代码链路；4. 提炼可迁移设计模式	产出一版可接入 Agently ModelRequest 的 runtime 模块	学员从“理解 runtime”进入“能够抽象 runtime 设计”	Runtime 作为服务核心内核，对外只暴露稳定接口；内部通过 hooks / registry 扩展

第 7 章：Skills 类型、供应与消费机制（3 课）

节（每节课约 2 小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
第 7-1 课： Skills 类型、调用方式与系统位置	1. 对照 Codex、Claude Code 与课程项目中的 Skills 机制；2. 区分 reference 型、任务型、script 型、MCP 依赖型 Skill；3. 区分显式调用、隐式匹配、上下文内联、编排流程消费等几种调用方式；4. 明确 Skills 与 prompt、tool、MCP、workflow 的边界	先定义 1 个 reference 型 Skill、1 个 script 型 Skill、1 个 MCP 依赖型 Skill 的最小骨架	学员理解当前常见 Skills 的主要类型，以及不同类型在系统中如何被触发和消费	Skill 是资产层，调用策略、依赖声明和上下文加载方式必须显式建模
第 7-2 课： Skills 供应侧：撰写、调试与管理	1. 讲清 Skill 描述、触发条件、参数提示、依赖声明和权限边界如何撰写；2. 说明 references、scripts、assets、MCP 依赖在 Skill 中各自承担什么角色；3. 演示 Skills 的调试、回归样例、版本管理和 registry 管理；4. 讨论团队协作中的 Skill 评审、发布和淘汰机制	补齐三类 Skill 的内容、依赖和测试样例，形成第一版 Skill registry	学员能够从零撰写、调试并管理多类型 Skill 资产	供应侧要把内容、依赖、权限、版本、归属和测试一起管理，不能只写一段说明文本
第 7-3 课： Skills 消费侧：解析、匹配与运行时接入	1. 设计 Skill 的发现、索引、解析与匹配流程；2. 讲清显式触发、规则匹配、描述匹配与依赖检查；3. 处理 Skill 上下文加载、工具绑定、MCP 绑定和沙盒权限；4. 把 Skill 接入 runtime 与 TriggerFlow，并补齐 fallback 与 trace	为课程项目实现最小 skill-parser、skill-registry、skill-loader 与 runtime 接入模块	学员能够实现一个可用的 Skills 解析与消费模块，而不是只会写 Skill 文件	供应侧和消费侧必须解耦；Skill 解析、匹配、装配不能散落在业务逻辑里

第 8 章：事件流、流式反馈与主动任务（2 课）

参考场景：长表单填写进度反馈、定时日报、审批提醒、内部流程节点推进

节（每节课约 2 小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
第 8-1 课：事件流、流式反馈与主动触发	1. 展示参考案例中的 partial、tool event、status event 与主动触发过程；2. 明确仅有最终结果的局限；3. 说明事件对前端展示与运维排查的价值；4. 理解主动任务在智能体系统中的作用	给任务台增加执行过程展示，能看到步骤、状态、工具回调	学员理解“过程可观测”在智能体系统中的价值	前端事件流与内部审计事件要区分；统一事件 schema 才能支持 UI、日志和回放
第 8-2 课：事件总线与 TriggerFlow 主动流程	1. 统一事件结构；2. SSE / WebSocket 消费边界；3. cron、polling、heartbeat 的分工；4. 主动任务如何与 FastAPI 请求处理分离；5. 如何用 Agently TriggerFlow 组织定时汇总、审批提醒、任务推进这类内部工作流	使用 Agently TriggerFlow 完成一个最小事件总线 and 定时触发流程	学员能从“理解事件系统”走到“会设计最小事件系统”	事件总线、scheduler、runtime 的边界分清；主动任务不直接耦合在 API 请求里

第 9 章：状态管理、可控记忆与任务恢复

(2 课)

参考场景：用户偏好资料、历史申请记录、跨轮表单补全、常用内部查询条件

节（每节课约 2 小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
第 9-1 课：Session、Transcript 与短期状态	1. 展示参考案例中的 session、transcript 和上下文裁剪现象；2. 说明 session 的意义；3. 梳理短期上下文的保留与压缩方式；4. 明确 session 不能简单等同于聊天记录	为服务增加短期状态保存、裁剪和基础摘要	学员理解短期状态与会话边界是系统设计问题	运行态上下文与持久化状态分层；session store 不直接和 UI 展示格式绑定
第 9-2 课：可控记忆、显式写回与任务恢复	1. 区分用户资料、任务草稿、知识库、检索库和真正意义上的“记忆”；2. 说明企业场景中哪些资料适合写回，哪些不适合交给模型自动长期沉淀；3. 设计经确认写回、schema 化资料存储与人工审核机制；4. 把 checkpoint、resume、replay 与任务恢复合并成同一套状态管理设计	为服务增加 profile store、显式写回接口、checkpoint、resume 和任务回放	学员理解企业里真正可落地的“记忆”应该如何收敛为状态管理与可控写回	检索不等于记忆；自动长期记忆默认弱化，优先采用 schema 化、可审核、可回滚的状态写回策略

第 10 章：Eval、回归测试与可观测性

(2 课)

参考场景：FAQ 答复正确率、表单字段完整性、流程节点选择准确性

节（每节课约 2 小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
第 10-1 课：Eval 样例集与回归检查	1. 以 FAQ 答错、字段漏填、流程节点选错等问题为例，说明缺少 eval / regression 的风险；2. 说明任务样例集、基线结果、通过标准的定义；3. 区分离线评测、在线检查与人工验收的边界；4. 梳理规则校验与模型评判的分工	为服务增加最小 eval 样例集、回归检查任务和结果记录	学员能判断一次改动到底是“更好了”还是“只是看起来能跑”	评测契约独立于任务执行模块；输入样例、期望结果、判定逻辑分别管理
第 10-2 课：Trace、回放与可观测体系	1. task_id、trace_id、step_id 的设计；2. 区分事件、日志、指标三类观测数据的分工；3. 明确回放能力在排障和复盘中的作用；4. 说明如何把执行轨迹和评测结果串联起来	给服务增加 trace 字段、关键节点埋点和回放视图	学员能从“看到结果”升级到“能定位问题出在哪一层”	统一事件 schema，打通 UI 展示、日志排查和 eval 复盘

第 11 章：成本、可靠性、安全基线与服务化收束（2 课）

参考场景：高频内部问答、重复表单提交、批量流程触发

节（每节课约 2 小时）	课程核心内容	课程项目演进	技能 / 认知成长收益	架构思想收益
第 11-1 课：成本控制与可靠性设计	1. 以高频内部问答、内部门户表单操作、批量流程触发等场景为例，展示成本失控与失败重试问题；2. 区分 budget、timeout、retry、fallback、degrade 的职责；3. 设计并发、排队、幂等、失败重试的	给服务增加预算字段、超时策略、重试规则和幂等键	学员能把“系统跑起来”推进到“系统连续跑得住”	成本控制与失败策略要前置到 runtime、worker、gateway，而不是上线后补洞

节（每 节课约 2 小时）	课程核心内容	课程项目 演进	技能 / 认知成 长收益	架构思想收益
	最小方案；4. 理解智能体系统对质量与成本的双重约束			
第 11-2 课：安 全基 线、服 务化拓 扑与课 程收束	1. 最小权限、执行审批、审计记录、人工接管；2. 密钥、配置、执行权限的边界；3. 本地 PoC 到服务化部署的最小拓扑；4. 课程总复盘与后续扩展方向	完成通用智能助理服务最终 PoC 方案、演示路径和服务拓扑图	学员能够把前面学到的模块收束成一套可继续扩展的服务结构	Gateway、Runtime、Worker、Memory、Eval、Ops 的职责统一收束；形成可以继续工程化演进的骨架

五、课程组织路径

1. 案例引入、原理讲解、模块实践与系统集成

- 各章按“案例演示 -> 原理讲解 -> 模块实践 -> 系统集成”展开。

2. 参考案例与课程项目双线并进

- 参考案例用于建立系统感知和模块映射。
- 课程项目用于完成 **Agently ModelRequest + Agently TriggerFlow + FastAPI + FastMCP** 的模块实现、集成和服务化落地。

3. 环境准备与框架初始化前置

- 第 0 章完成 OpenClaw 安装运行、Codex / Claude Code 演示，以及 **Agently ModelRequest + Agently TriggerFlow + FastAPI** 最小工程骨架搭建。
- 后续相关章节补充 **FastMCP**、Sandbox、Trace、Eval、**Agently TriggerFlow** 等专项环境。

4. 从单次请求到基础编排，再逐步扩展

- 课程先用 **Agently ModelRequest** 完成单次请求，再用 **Agently TriggerFlow** 建立基础编排方式，随后扩展结构化决策、工具接入、沙盒、FastAPI 服务、runtime、skills、事件、状态管理和治理能力。

5. 按具体技术栈推进系统搭建

- 课程实现顺序为：**OpenClaw / Codex / Claude Code** 演示 -> **Agently ModelRequest** -> **Agently TriggerFlow** 基础编排 -> 工具与 **FastMCP** -> 沙盒 -> **FastAPI** -> **Runtime** -> **Skills** -> **TriggerFlow** 事件化编排 -> 状态管理 / 可控记忆 -> **Eval** / 观测 -> 安全与服务化。

6. 企业内部场景贯穿全程

- 各章案例持续围绕企业内部问答、检索、长文梳理、内网接口访问、流程协同与 workflow 编排展开。

六、课程交付成果

- 一套可安装、可运行、可观察的 **OpenClaw**、**Codex**、**Claude Code** 参考演示环境。
- 一个基于 **Agently ModelRequest** + **Agently TriggerFlow** + **FastAPI** 的本地可运行通用智能助理服务。
- 一个可提交任务并查看过程事件的简易任务台。
- 一套最小可用的 **planner**、**tool registry**、**FastMCP Server**、**sandbox**、**runtime** 模块。
- 一套包含 **reference** 型、**script** 型、**MCP** 依赖型 **Skill** 的资产组织方式，以及对应的解析与消费模块。
- 一套基于 **Agently TriggerFlow** 的基础编排与主动流程实现，以及 **session**、**显式写回**、**checkpoint**、**resume** 的基础设计。
- 一套最小 **eval** 样例集、回归检查脚本与 **trace / replay** 机制。
- 一套预算、超时、重试、幂等与最小安全基线设计。
- 一张可继续演进的服务化系统架构图。